

Comparative Analysis of Graph Traversal and Shortest Path Algorithms

Sumit Desai

Department of Computer Science & Engineering

COEP Technological University

Pune, India

MIS: 612415049

Abstract—Graph theory provides the fundamental mathematical structures used to model pairwise relationships in complex systems, ranging from social networks and biological pathways to transportation grids and telecommunication networks. The efficiency of navigating these structures is paramount to system performance. This paper presents a rigorous comparative analysis of three foundational graph algorithms: Breadth-First Search (BFS), Depth-First Search (DFS), and Dijkstra’s Algorithm. We examine the theoretical underpinnings of their traversal logic, analyze their time and space complexities based on graph representation choices, and evaluate their suitability for specific problem domains, particularly focusing on shortest path determination in unweighted versus weighted graphs. Furthermore, we mathematically define the conditions under which Dijkstra’s greedy approach yields optimal results versus where unweighted traversal suffices. The results highlight critical trade-offs: while BFS and DFS offer linear time complexity for basic traversal, Dijkstra’s algorithm, despite higher computational overhead, is indispensable for cost-sensitive pathfinding in non-negative weighted graphs.

Index Terms—Graph Theory, Pathfinding, Computational Complexity, Adjacency List, Priority Queue, BFS, DFS, Dijkstra

I. INTRODUCTION

A graph $G = (V, E)$ is a versatile data structure consisting of a set of vertices V and a set of edges E that connect pairs of vertices. The immense utility of graphs stems from their ability to abstractly represent diverse real-world scenarios. For example, in computer networking, vertices may represent routers and edges the physical links between them; in social media, vertices represent users and edges their relationships.

A fundamental operation on graphs is traversal—the process of visiting (checking and/or updating) each vertex in a graph, exactly once. Traversal forms the basis for many complex algorithms, including connectivity analysis, cycle detection, and, crucially, finding the shortest path between two nodes.

The “shortest path” problem varies significantly depending on the nature of the graph’s edges.

- **Unweighted Graphs:** The shortest path is defined by the minimum number of edges between a source and a destination.
- **Weighted Graphs:** Each edge has an associated numerical value (weight/cost), and the shortest path is the path with the minimum cumulative weight.

This paper analyzes standard approaches to these problems. We compare the uninformed search strategies of Breadth-First Search (BFS) and Depth-First Search (DFS) against the informed, greedy strategy of Dijkstra’s algorithm. We analyze why certain algorithms perform optimally in specific graph topologies while failing in others, providing a theoretical basis for algorithm selection in software engineering contexts.

II. METHODOLOGY

Our methodology involves a theoretical analysis of algorithm mechanics followed by an evaluation of their computational constraints based on data structure selection.

A. Graph Representation and Complexity

The choice of underlying data structure profoundly impacts the performance of graph algorithms. We consider a graph with $|V|$ vertices and $|E|$ edges.

1) *Adjacency Matrix:* A 2D array of size $|V| \times |V|$ where cell (i, j) indicates an edge between vertex i and j . While checking for edge existence is $O(1)$, the space complexity is quadratically expensive, making it unsuitable for sparse real-world graphs.

$$Space_{Matrix} = O(|V|^2) \quad (1)$$

2) *Adjacency List:* An array of lists where index i contains a list of all vertices adjacent to vertex i . This representation is space-efficient for sparse graphs, where $|E| \ll |V|^2$.

$$Space_{List} = O(|V| + |E|) \quad (2)$$

Given the sparsity of most practical networks, our analysis assumes the use of an Adjacency List representation, as defined in Equation (2).

B. Algorithm Mechanics

1) *Breadth-First Search (BFS):* BFS is a traversal strategy that explores vertices layer by layer. Starting from a source vertex s , it first visits all neighbors of s , then all neighbors of those neighbors, and so on. This radiating exploration is implemented using a First-In-First-Out (FIFO) **Queue** data structure.

Because BFS explores layers equidistantly, in an unweighted graph, the first time BFS reaches a target node, it is guaranteed to have taken the shortest path (fewest edges). Fig. 1 illustrates this concentric expansion.

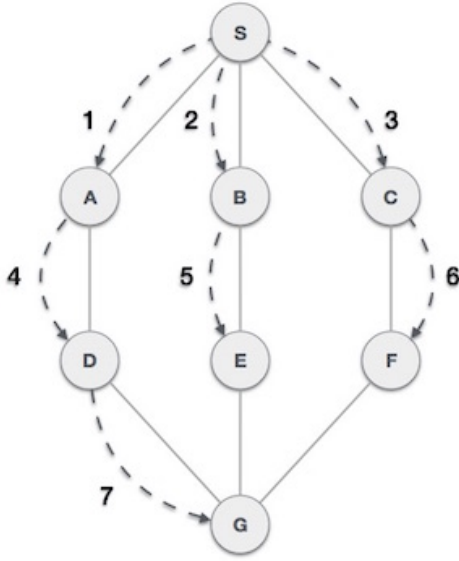


Fig. 1. BFS traversal spreading concentrically from source node 'S'. Nodes are numbered by the order they are visited.

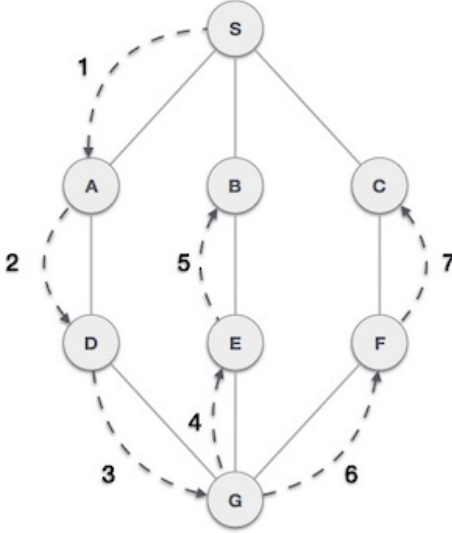


Fig. 2. DFS traversal showing deep exploration along one branch before backtracking.

2) *Depth-First Search (DFS)*: DFS is an aggressive traversal strategy. From a source node, it explores as far as possible along a single branch before backtracking. This approach is implemented using a Last-In-First-Out (LIFO) **Stack** (often implicitly via system recursion stack).

DFS is not suitable for shortest path findings, as it may traverse a very long path to a neighbor that was actually reachable via a single direct edge. However, its backtracking nature makes it ideal for topological sorting and detecting cycles. Fig. 2 demonstrates the deep plunging nature of DFS.

3) *Dijkstra's Algorithm*: Dijkstra's algorithm solves the single-source shortest path problem for graphs with non-negative edge weights. It employs a greedy approach, main-

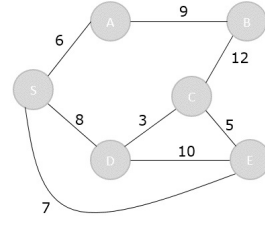


Fig. 3. Dijkstra's Algorithm. The shortest path from S to C is 11 (cumulative cost) i.e path $S \rightarrow D \rightarrow C$.

taining a set of unvisited vertices and, at each step, selecting the unvisited vertex with the smallest tentative distance from the source.

This selection process requires a **Min-Priority Queue**. The core of the algorithm is the "relaxation" process. For an edge (u, v) with weight $w(u, v)$, if the current known distance to v ($d[v]$) is greater than the distance to u plus the weight of the edge between them, we update $d[v]$:

$$d[v] = \min(d[v], d[u] + w(u, v)) \quad (3)$$

Equation (3) ensures that we always store the lowest cost path found so far. Fig. 3 shows Dijkstra selecting paths based on cumulative weight rather than just edge count.

III. RESULTS AND COMPARATIVE ANALYSIS

A. Time Complexity Analysis

For BFS and DFS, every vertex and every edge is processed exactly once in the adjacency list representation. Thus, their time complexity is linear relative to the graph size: $O(|V| + |E|)$.

Dijkstra's complexity is dominated by priority queue operations. Using a standard binary heap, extracting the minimum element takes $O(\log |V|)$ time. This occurs once per vertex. Additionally, edge relaxation may require decreasing a key in the priority queue, which also takes $O(\log |V|)$ time, potentially happening for every edge. The resulting complexity is $O(|E| \log |V|)$.

B. Space Complexity and Memory Usage

While all utilize $O(|V| + |E|)$ to store the graph itself, their auxiliary space requirements during execution differ, as summarized in Table I.

- **BFS**: The queue can grow to the size of the largest layer in the graph. In the worst case (a star graph), this is $O(|V|)$.
- **DFS**: The recursion stack grows to the maximum depth of the graph. In a skewed, line-like graph, this is $O(|V|)$. This can lead to stack overflow issues in very deep graphs.
- **Dijkstra**: The priority queue stores distance estimates for all vertices, requiring $O(|V|)$ auxiliary space.

TABLE I
COMPARATIVE SUMMARY OF GRAPH ALGORITHMS

Metric	BFS	DFS	Dijkstra
Core Structure	FIFO Queue	LIFO Stack	Priority Queue
Time Compl.	$O(V + E)$	$O(V + E)$	$O(E \log V)$
Aux. Space	$O(V)$ (Width)	$O(V)$ (Depth)	$O(V)$
Shortest Path?	Yes (Unweighted)	No	Yes (Weighted)
Optimal Use	Networking broadcast, peer discovery.	Cycle detection, maze generation.	GPS routing, network routing protocols.

C. Limitations

It is crucial to note that Dijkstra’s algorithm fails if the graph contains negative edge weights. The greedy assumption—that once a vertex is “visited” its optimal distance is finalized—is violated by negative edges, which could subsequently reduce the total path cost. In such scenarios, the Bellman-Ford algorithm is required, albeit at a higher time complexity of $O(|V||E|)$.

IV. CONCLUSION

The selection of an appropriate graph algorithm is strictly dependent on the graph’s properties and the specific problem constraint. Through our analysis, we conclude that BFS is the optimal choice for shortest path problems on unweighted graphs due to its linear time complexity and layer-wise exploration. DFS, while computationally equivalent to BFS, is structurally unsuited for shortest paths but excels in connectivity and backtracking scenarios. Dijkstra’s algorithm remains the industry standard for weighted graphs with non-negative edges, trading increased computational complexity (due to priority queue management defined in Equation (3)) for the ability to handle varying edge costs. Modern large-scale systems often employ hybrid approaches, using BFS for localized searches and Dijkstra or A* variants for long-distance routing.

REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009, pp. 594–663.
- [2] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2011.
- [3] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [4] TutorialsPoint, “Graph traversal algorithms,” TutorialsPoint, Online. Available: TutorialsPoint.com